C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory, Edge Cases & Code Examples

- ✓ The Spaceship Operator
- ✓ strong, weak, partial
- ✓ Defaulted and Custom `operator<=>`
- ✓ `operator==` vs `operator<=>`
- ✓ Designated Initializers

→ *Swipe to learn* →

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 1. Three-Way Comparison Operator (`<=>`) - The Spaceship Operator (1/2)

The Problem Before C++20:

```
struct Point {
    int x, y;

    // ❌ C++17: Need to define all 6 comparison operators manually
    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
    bool operator!=(const Point& other) const {
        return !(*this == other);
    }
    bool operator<(const Point& other) const {
        if (x < other.x) return true;
        if (x > other.x) return false;
        return y < other.y;
    }
    bool operator<=(const Point& other) const {
        return !(other < *this);
    }
    bool operator>(const Point& other) const {
        return other < *this;
    }
    bool operator>=(const Point& other) const {
        return !(*this < other);
    }
};
// 6 functions, error-prone, boilerplate
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 1. Three-Way Comparison Operator (`<=>`) - The Spaceship Operator (2/2)

The C++20 Solution:

```
struct Point {
    int x, y;

    // ✅ C++20: Define operator<=> and optionally operator==
    auto operator<=>(const Point& other) const = default;
};

// Automatically generates all 6 comparison operators!
Point p1{1, 2}, p2{3, 4};
p1 == p2;  // ✅ Available
p1 != p2;  // ✅ Available
p1 < p2;   // ✅ Available
p1 <= p2;  // ✅ Available
p1 > p2;   // ✅ Available
p1 >= p2;  // ✅ Available
```

How `<=>` Works:

The spaceship operator returns one of three comparison category types:

```
auto result = a <=> b;

// result can be:
// - std::strong_ordering::{less, equal, greater}
// - std::weak_ordering::{less, equivalent, greater}
// - std::partial_ordering::{less, equivalent, greater, unordered}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 2. Comparison Categories - strong, weak, partial (1/2)

std::strong_ordering:

Values are **fully ordered** and **substitutable**:
- `a == b` means `a` and `b` are indistinguishable
- Used for types like `int`, `std::string`, pointers

```
#include <compare>

struct Integer {
    int value;

    std::strong_ordering operator<=>(const Integer& other) const {
        return value <=> other.value;  // int has strong ordering
    }
};

Integer a{5}, b{10};
auto result = a <=> b;  // std::strong_ordering::less
```

std::weak_ordering:

Values are **fully ordered** but **not necessarily substitutable**:
- `a == b` means they're equivalent, but may differ in unobserved ways
- Used for case-insensitive strings, normalized forms

```
struct CaseInsensitiveString {
    std::string value;

    std::weak_ordering operator<=>(const CaseInsensitiveString& other) const {
        return to_lower(value) <=> to_lower(other.value);
        // "Hello" and "HELLO" are equivalent (not equal in strong sense)
    }
};
```

std::partial_ordering:

Some values may be **unordered** (not comparable):
- Used for floating-point (NaN is unordered)
- Partially ordered sets

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 2. Comparison Categories - strong, weak, partial (2/2)

```
double a = 1.0, b = std::numeric_limits<double>::quiet_NaN();
auto result = a <=> b;  // std::partial_ordering::unordered

// NaN is unordered with everything
if (result == std::partial_ordering::unordered) {
    std::cout << "Cannot compare\n";
}
```

Comparison Table:

Category	Substitutable	All Values Comparable	Examples
`strong_ordering`	✅ Yes	✅ Yes	`int`, `std::string`
`weak_ordering`	❌ No	✅ Yes	Case-insensitive strings
`partial_ordering`	❌ No	❌ No	`float`, `double` (NaN)

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 3. Defaulted and Custom `operator<=>` (1/2)

Defaulted (Compiler-Generated):

```
struct Person {
    std::string name;
    int age;

    // Compiler generates memberwise comparison (name first, then age)
    auto operator<=>(const Person&) const = default;
};

// Generated comparison:
// 1. Compare name (std::string has strong_ordering)
// 2. If equal, compare age (int has strong_ordering)
// Result: std::strong_ordering
```

Custom Implementation:

```
struct Product {
    std::string name;
    double price;

    std::strong_ordering operator<=>(const Product& other) const {
        // Custom logic: Compare by price first, then name
        if (auto cmp = price <=> other.price; cmp != 0)
            return cmp;
        return name <=> other.name;
    }
};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 3. Defaulted and Custom `operator<=>` (2/2)

Return Type Deduction:

```
struct Data {
    int a;
    double b;  // partial_ordering (due to NaN)

    // Return type is auto (deduced as partial_ordering)
    auto operator<=>(const Data&) const = default;
};

// Compiler chooses "weakest" category among members:
// int → strong_ordering
// double → partial_ordering
// Result → partial_ordering (weakest wins)
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 4. `operator==` vs `operator<=>`

Key Insight: `operator==` is NOT automatically generated from `operator<=>` for efficiency!

```
struct Point {
    int x, y;

    auto operator<=>(const Point&) const = default;  // Generates <, <=, >, >=
    bool operator==(const Point&) const = default;   // Generates ==, !=
};

// Why separate?
// operator<=> might be expensive (compute ordering)
// operator== can be optimized (just check equality, no ordering)
```

Example:

```
struct BigData {
    std::vector<int> data;

    // Efficient equality check
    bool operator==(const BigData& other) const {
        return data.size() == other.size() &&
               std::equal(data.begin(), data.end(), other.begin());
    }

    // More expensive ordering
    auto operator<=>(const BigData& other) const {
        return data <=> other.data;  // Lexicographic comparison
    }
};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 5. Designated Initializers (1/2)

The Feature:

Initialize aggregate members by name in any order:

```
struct Config {
    int port;
    std::string host;
    bool ssl;
    int timeout;
};

// ✅ C++20: Clear, self-documenting
Config cfg {
    .port = 8080,
    .host = "localhost",
    .ssl = true,
    .timeout = 30
};

// Can omit members (default-initialized)
Config cfg2 {
    .port = 443,
    .ssl = true
    // host = "", timeout = 0
};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 5. Designated Initializers (2/2)

Rules:

1. **Order must match declaration order:**
   ```cpp
   struct Point { int x, y; };

// ✅ OK: x before y
Point p{.x = 1, .y = 2};

// ❌ Error: y before x
Point p{.y = 2, .x = 1};
```

1. **Cannot mix with positional initialization:**
   `cpp // ❌ Error: Can't mix Config cfg{8080, .ssl = true};`
2. **Nested initialization:**
   ```cpp
   struct Address {
   std::string city;
   int zip;
   };

struct Person {
std::string name;
Address addr;
};

Person p {
.name = "Alice",
.addr = {.city = "NYC", .zip = 10001}
};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 6. `consteval` - Immediate Functions (1/2)

Difference from `constexpr`:

Feature	`constexpr`	`consteval`
Must run at compile-time?	No (can run at runtime)	**Yes** (always compile-time)
Can return non-constexpr?	Yes	No
Use case	Optionally compile-time	Guaranteed compile-time

consteval Example:

```
consteval int square(int x) {
    return x * x;
}

int main() {
    constexpr int a = square(5);  // ✅ OK: Compile-time
    int b = square(10);           // ✅ OK: Still compile-time

    int n;
    std::cin >> n;
    int c = square(n);            // ❌ Error: n is runtime variable
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 6. `consteval` - Immediate Functions (2/2)

Use Cases:

1. **Meta-programming that must happen at compile-time:**
   `cpp consteval auto get_type_name() { return "int"; // Must be evaluated at compile-time }`
2. **Enforce compile-time computation:**
   ```cpp
   consteval size_t compute_buffer_size(size_t elements) {
   return elements * sizeof(int);
   }

// Buffer size is guaranteed known at compile-time
std::array buffer;
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 7. `constinit` - Compile-Time Initialization

The Problem:

Static initialization order fiasco:

```
// ❌ C++17: Initialization order undefined across translation units
int global1 = compute1();  // When is this initialized?
int global2 = compute2();  // What if compute2() uses global1?
```

constinit Solution:

Guarantees variable is initialized at compile-time (or during constant initialization):

```
// ✅ C++20: Guaranteed compile-time initialization
constinit int global1 = 42;
constinit int global2 = global1 * 2;  // Safe: global1 already initialized
```

Rules:

```
constinit int a = 10;       // ✅ OK: Compile-time constant
constinit int b = a * 2;    // ✅ OK: Uses compile-time value

int runtime_func() { return 42; }
constinit int c = runtime_func();  // ❌ Error: Not compile-time
```

constinit vs constexpr:

```
constexpr int a = 42;  // Compile-time constant, can't be modified
a = 100;               // ❌ Error: Can't modify constexpr

constinit int b = 42;  // Compile-time initialization, but mutable
b = 100;               // ✅ OK: Can modify after initialization
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 8. [[likely]] and [[unlikely]] Attributes (1/2)

Purpose: Hint to compiler which branch is more probable for optimization.

Syntax:

```
void process(int value) {
    if (value > 0) [[likely]] {
        // This branch is expected to be taken most of the time
        fast_path(value);
    } else [[unlikely]] {
        // This branch is rarely taken
        slow_error_handling();
    }
}
```

Impact:

```
// Without attributes:
if (rare_error) {  // Compiler doesn't know this is rare
    handle_error();
}
process_normal();

// With attributes:
if (rare_error) [[unlikely]] {  // Compiler optimizes for unlikely case
    handle_error();
}
process_normal();  // This path is optimized as the common case
```

Switch Statements:

```
switch (operation) {
    case OP_ADD: [[likely]]
        return a + b;

    case OP_SUBTRACT:
        return a - b;

    case OP_ERROR: [[unlikely]]
        throw std::runtime_error("Error");
}
```

📌 8. [[likely]] and [[unlikely]] Attributes (2/2)

Performance:

- Improves branch prediction
- Better instruction cache utilization
- Can reduce mispredicted branches by 10-30% in hot paths

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 9. `using enum` - Enum Scope Reduction

The Problem:

```
enum class Color { Red, Green, Blue };

void paint(Color c) {
    // ❌ C++17: Verbose, repetitive
    switch (c) {
        case Color::Red:   /* ... */ break;
        case Color::Green: /* ... */ break;
        case Color::Blue:  /* ... */ break;
    }
}
```

C++20 Solution:

```
void paint(Color c) {
    using enum Color;  // ✅ Bring all enumerators into scope

    switch (c) {
        case Red:   /* ... */ break;
        case Green: /* ... */ break;
        case Blue:  /* ... */ break;
    }
}
```

Selective Import:

```
void use_colors() {
    using Color::Red;   // Import only Red
    using Color::Blue;  // Import only Blue

    auto c1 = Red;      // ✅ OK
    auto c2 = Blue;     // ✅ OK
    auto c3 = Green;    // ❌ Error: Not imported
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 10. Template Improvements

Template Lambda:

```
// ✅ C++20: Lambda with explicit template parameter
auto lambda = []<typename T>(T value) {
    std::cout << "Type: " << typeid(T).name() << '\n';
    return value * 2;
};

lambda(42);      // T = int
lambda(3.14);    // T = double
```

Concepts in Templates:

```
template<std::integral T>  // ✅ C++20: Concepts as template parameters
T add(T a, T b) {
    return a + b;
}
```

Non-Type Template Parameters:

```
// ✅ C++20: Floating-point and class types as template parameters
template<double Ratio>
double scale(double value) {
    return value * Ratio;
}

auto scaled = scale<1.5>(100.0);  // 150.0
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 11. Aggregate Improvements

Direct-List-Initialization of Aggregates:

```
struct Point { int x, y; };

// ✅ C++20: Parenthesized initialization
Point p1(1, 2);     // Now allowed (was only p{1, 2} before)
Point p2 = Point(3, 4);
```

Aggregate with Base Class:

```
struct Base { int a; };
struct Derived : Base { int b; };

// ✅ C++20: Can initialize base in aggregate
Derived d{.a = 1, .b = 2};  // Initialize Base::a and Derived::b
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Theory

📌 12. Conditionally `explicit` Constructors

The Feature:

```
template<typename T>
struct Optional {
    T value;

    // ✅ C++20: Conditionally explicit based on T
    template<typename U>
    explicit(!std::is_convertible_v<U, T>)
    Optional(U&& val) : value(std::forward<U>(val)) {}
};

Optional<int> o1 = 42;        // OK if int is convertible
Optional<MyClass> o2 = obj;   // explicit if MyClass not convertible
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Edge Cases

📌 1. Mixed Comparison Categories

Problem:

```
struct Mixed {
    int a;           // strong_ordering
    double b;        // partial_ordering (NaN)

    auto operator<=>(const Mixed&) const = default;
};

// What's the return type?
// Answer: partial_ordering (weakest wins)
```

Rule: Compiler selects the **weakest** category among all members.

C++20 Language Features - Three-Way Comparison and Modern Syntax

Edge Cases

📌 2. Base Class Comparison

Problem:

```
struct Base {
    int x;
    auto operator<=>(const Base&) const = default;
};

struct Derived : Base {
    int y;
    auto operator<=>(const Derived&) const = default;  // ✅ Includes Base::x
};

Derived d1{.x=1, .y=2}, d2{.x=1, .y=3};
auto cmp = d1 <=> d2;  // Compares Base::x first, then Derived::y
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Edge Cases

📌 3. `operator==` Not Synthesized from `operator<=>`

Problem:

```
struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;  // Only generates <, <=, >, >=
};

Point p1{1, 2}, p2{1, 2};
bool eq = (p1 == p2);  // ❌ Error: operator== not defined!

// Solution:
struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;
    bool operator==(const Point&) const = default;  // ✅ Add this
};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Edge Cases

📌 4. Designated Initializers Order Matters

Problem:

```
struct Data { int a, b, c; };

// ❌ Error: Designators out of order
Data d{.b = 2, .a = 1, .c = 3};

// ✅ OK: Correct order
Data d{.a = 1, .b = 2, .c = 3};
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Edge Cases

📌 5. `consteval` Propagation

Problem:

```
consteval int compute() { return 42; }

constexpr int use_compute() {
    return compute();  // ❌ Error: constexpr calls consteval
}

// Solution:
consteval int use_compute() {  // ✅ Make it consteval too
    return compute();
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 1. Spaceship Operator for Custom Type

```
#include <compare>
#include <string>
#include <iostream>

struct Person {
    std::string name;
    int age;

    // Custom spaceship: Compare by age, then name
    std::strong_ordering operator<=>(const Person& other) const {
        if (auto cmp = age <=> other.age; cmp != 0)
            return cmp;
        return name <=> other.name;
    }

    // Also define equality
    bool operator==(const Person& other) const = default;
};

int main() {
    Person p1{"Alice", 30}, p2{"Bob", 25}, p3{"Alice", 30};

    std::cout << std::boolalpha;
    std::cout << (p1 <=> p2 == std::strong_ordering::greater) << '\n';  // true (30 > 25)
    std::cout << (p1 == p3) << '\n';  // true
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 2. Designated Initializers for Configuration

```
#include <string>
#include <iostream>

struct ServerConfig {
    std::string host = "localhost";
    int port = 8080;
    bool use_ssl = false;
    int timeout_ms = 5000;
    int max_connections = 100;
};

int main() {
    // Override only what you need
    ServerConfig dev_config {
        .port = 3000,
        .timeout_ms = 10000
    };

    ServerConfig prod_config {
        .host = "prod.example.com",
        .port = 443,
        .use_ssl = true,
        .max_connections = 1000
    };

    std::cout << "Dev: " << dev_config.host << ":" << dev_config.port << '\n';
    std::cout << "Prod: " << prod_config.host << ":" << prod_config.port << '\n';
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 3. `consteval` for Compile-Time Validation

```
#include <array>

consteval size_t validate_size(size_t n) {
    if (n == 0 || n > 1024)
        throw "Invalid buffer size";  // Compile-time error if condition met
    return n;
}

template<size_t N>
class Buffer {
    std::array<char, validate_size(N)> data_;  // Validated at compile-time
};

int main() {
    Buffer<100> b1;   // ✅ OK
    // Buffer<0> b2;     // ❌ Compile error: Invalid buffer size
    // Buffer<2000> b3;  // ❌ Compile error: Invalid buffer size
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 4. Branch Prediction Hints

```
#include <iostream>
#include <random>

int process_value(int value) {
    if (value < 0) [[unlikely]] {
        std::cerr << "Negative value encountered\n";
        return -1;
    }

    // Hot path (expected case)
    return value * 2;
}

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(0, 1000);

    for (int i = 0; i < 1000000; ++i) {
        int val = dis(gen);
        process_value(val);  // Compiler optimizes for val >= 0
    }
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 5. `using enum` in Switch

```
#include <iostream>

enum class Operation { Add, Subtract, Multiply, Divide };

double calculate(Operation op, double a, double b) {
    using enum Operation;  // Bring enumerators into scope

    switch (op) {
        case Add:      return a + b;
        case Subtract: return a - b;
        case Multiply: return a * b;
        case Divide:   return b != 0 ? a / b : 0;
    }
    return 0;
}

int main() {
    std::cout << calculate(Operation::Add, 10, 5) << '\n';       // 15
    std::cout << calculate(Operation::Multiply, 10, 5) << '\n';  // 50
}
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 6. Template Lambda with Explicit Template Parameters

Output:

```
#include <iostream>
#include <vector>
#include <string>

int main() {
    // ✅ C++20: Lambda with explicit template parameter
    auto print_twice = []<typename T>(T value) {
        std::cout << value << " " << value << '\n';
    };

    print_twice(42);           // int: 42 42
    print_twice(3.14);         // double: 3.14 3.14
    print_twice("Hello");      // const char*: Hello Hello

    // Advanced: Use in algorithms
    auto doubler = []<typename T>(const T& x) { return x + x; };

    std::vector<int> nums{1, 2, 3};
    for (const auto& n : nums) {
        std::cout << doubler(n) << ' ';  // 2 4 6
    }
}
```

```
['42 42\n3.14 3.14\nHello Hello\n2 4 6']
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 7. `constinit` for Safe Global Initialization

Output:

```
#include <iostream>

// ❌ Problem without constinit: Static initialization order fiasco
int compute_value() { return 42; }
// int global1 = compute_value();  // When is this initialized?

// ✅ Solution with constinit
constinit int global1 = 42;           // Compile-time init
constinit int global2 = global1 * 2;  // Safe: global1 already exists

// Can still modify after initialization
void modify_globals() {
    global1 = 100;  // ✅ OK: constinit variables are mutable
    global2 = 200;
}

int main() {
    std::cout << "Initial: " << global1 << ", " << global2 << '\n';
    modify_globals();
    std::cout << "Modified: " << global1 << ", " << global2 << '\n';
}
```

```
['Initial: 42, 84\nModified: 100, 200']
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 8. `partial_ordering` with NaN

Output:

```
#include <compare>
#include <iostream>
#include <limits>

struct FloatWrapper {
    double value;

    auto operator<=>(const FloatWrapper& other) const {
        return value <=> other.value;  // Returns partial_ordering
    }
};

int main() {
    FloatWrapper a{1.0};
    FloatWrapper b{2.0};
    FloatWrapper nan_val{std::numeric_limits<double>::quiet_NaN()};

    auto cmp1 = a <=> b;
    if (cmp1 < 0) {
        std::cout << "1.0 < 2.0\n";  // This executes
    }

    auto cmp2 = a <=> nan_val;
    if (cmp2 == std::partial_ordering::unordered) {
        std::cout << "1.0 and NaN are unordered\n";  // This executes
    }

    // NaN is not equal to anything, including itself
    auto cmp3 = nan_val <=> nan_val;
    if (cmp3 == std::partial_ordering::unordered) {
        std::cout << "NaN is not even equal to itself\n";  // This executes
    }
}
```

```
['1.0 < 2.0\n1.0 and NaN are unordered\nNaN is not even equal to itself']
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 9. `weak_ordering` for Case-Insensitive Comparison

Output:

```
#include <compare>
#include <iostream>
#include <string>
#include <algorithm>

struct CaseInsensitiveString {
    std::string value;

    // Helper: Convert to lowercase
    static std::string to_lower(std::string s) {
        std::transform(s.begin(), s.end(), s.begin(), ::tolower);
        return s;
    }

    std::weak_ordering operator<=>(const CaseInsensitiveString& other) const {
        return to_lower(value) <=> to_lower(other.value);
    }

    bool operator==(const CaseInsensitiveString& other) const {
        return to_lower(value) == to_lower(other.value);
    }
};

int main() {
    CaseInsensitiveString s1{"Hello"};
    CaseInsensitiveString s2{"HELLO"};
    CaseInsensitiveString s3{"World"};

    std::cout << std::boolalpha;
    std::cout << "Hello == HELLO: " << (s1 == s2) << '\n';  // true (equivalent)
    std::cout << "Hello < World: " << (s1 < s3) << '\n';    // true

    // But the strings are not identical (weak, not strong)
    std::cout << "Actual strings: " << s1.value << " vs " << s2.value << '\n';
}
```

```
['Hello == HELLO: true\nHello < World: true\nActual strings: Hello vs HELLO']
```

C++20 Language Features - Three-Way Comparison and Modern Syntax

Code Examples

📌 10. Aggregate Improvements - Parenthesized Initialization

Output:

```
#include <iostream>
#include <string>

struct Point {
    int x;
    int y;
};

struct Person {
    std::string name;
    int age;
};

// Aggregate with base class (C++20 enhancement)
struct Base {
    int id;
};

struct Derived : Base {
    std::string name;
};

int main() {
    // ✅ C++20: Parenthesized aggregate initialization
    Point p1(10, 20);           // Like p1{10, 20}
    Person person1("Alice", 30);

    // ✅ C++20: Designated init with base class
    Derived d{.id = 1, .name = "Bob"};

    std::cout << "Point: (" << p1.x << ", " << p1.y << ")\n";
    std::cout << "Person: " << person1.name << ", age " << person1.age << '\n';
    std::cout << "Derived: id=" << d.id << ", name=" << d.name << '\n';
}
```

```
['Point: (10, 20)\nPerson: Alice, age 30\nDerived: id=1, name=Bob']
```

What You Learned

Key Takeaways

- ✓ The Spaceship Operator
- ✓ strong, weak, partial
- ✓ Defaulted and Custom `operator<=>`
- ✓ `operator==` vs `operator<=>`

→ Swipe to learn →